Table of Contents

AEM368 Final Project. 1
Constants 1
Analysis Settings 1
Airplane Data 2
Main Analysis and Plotting 2
AI Use Statement 28

AEM368 Final Project.

Collin Atkins 4/29/2026

```
clear;
close all;
clc;
```

Constants

This structure stores the physical constants and the unit conversions.

```
constants = struct();

constants.G0_FT_S2 = 32.174;
constants.RHO_MSL_SLUG_FT3 = 0.002378;
constants.RE_FT = 2.0925e7;
constants.HP_TO_FT_LBF_S = 550.0;
constants.SEC_PER_HR = 3600.0;
constants.MIN_PER_HR = 60.0;
constants.FT_S_TO_KT = 0.592483801;
constants.FT_TO_NM = 1.0 / 6076.11549;
constants.RAD_TO_DEG = 180.0 / pi;
```

Analysis Settings

This structure stores the analysis grids and plotting settings.

```
analysis = struct();

analysis.altitude_grid_ft = 0.0:100.0:60000.0;
analysis.equivalent_speed_grid_ft_s = linspace(20.0, 1000.0, 4500);
analysis.weight_num_steps = 600;
analysis.figure_font_size = 11;
analysis.marker_size = 7;
analysis.point_altitude_labels = {'MSL', 'Half SC', 'SC'};
analysis.curve_colors = [
    0.0000, 0.4470, 0.7410; ...
    0.8500, 0.3250, 0.0980; ...
    0.4660, 0.6740, 0.1880];
```

```
analysis.drag_color = [0.1000, 0.1000, 0.1000];
analysis.stall_color = [0.4940, 0.1840, 0.5560];
```

Airplane Data

```
airplanes = define_airplanes();
```

Main Analysis and Plotting

This loop runs the analysis and plotting for each of the airplanes.

```
for airplane_ind = 1:numel(airplanes)
    performance = analyze_airplane( ...
        airplanes(airplane_ind), constants, analysis);

    print_summary(performance, constants);

    make_required_plots(performance, constants, analysis);
end

% This function contains the jet and propeller airplane data.
function airplanes = define_airplanes()

jet_airplane = struct();

jet_airplane.name = 'Jet Airplane';
jet_airplane.type = 'jet';
jet_airplane.wingspan_ft = 53.3;
jet_airplane.wing_area_ft2 = 318.0;
jet_airplane.takeoff_weight_lbf = 19815.0;
jet_airplane.fuel_weight_lbf = 7500.0;
jet_airplane.thrust_available_msl_lbf = 7300.0;
jet_airplane.shaft_power_msl_hp = NaN;
jet_airplane.propeller_efficiency = NaN;
jet_airplane.fuel_rate = 0.6;
jet_airplane.cd0 = 0.02;
jet_airplane.cl_max = 1.75;
jet_airplane.oswald_efficiency = 0.81;
jet_airplane.n_max = 1.5;
jet_airplane.service_ceiling_roc_ft_min = 500.0;

propeller_airplane = struct();

propeller_airplane.name = 'Propeller Airplane';
propeller_airplane.type = 'propeller';
propeller_airplane.wingspan_ft = 35.8;
propeller_airplane.wing_area_ft2 = 174.0;
propeller_airplane.takeoff_weight_lbf = 2950.0;
propeller_airplane.fuel_weight_lbf = 390.0;
propeller_airplane.thrust_available_msl_lbf = NaN;
propeller_airplane.shaft_power_msl_hp = 230.0;
propeller_airplane.propeller_efficiency = 0.8;
propeller_airplane.fuel_rate = 0.45;
```

```
propeller_airplane.cd0 = 0.025;
propeller_airplane.cl_max = 2.0;
propeller_airplane.oswald_efficiency = 0.8;
propeller_airplane.n_max = 1.5;
propeller_airplane.service_ceiling_roc_ft_min = 100.0;

airplanes = [jet_airplane, propeller_airplane];
end

% This function contains the airplane performance analysis.
function performance = analyze_airplane(airplane, constants, analysis)

performance = struct();

performance.airplane = airplane;
performance.aspect_ratio = airplane.wingspan_ft^2 / ...
    airplane.wing_area_ft2;
performance.induced_drag_factor = 1.0 / (pi * performance.aspect_ratio ...
    * airplane.oswald_efficiency);

performance.stall_eas_ft_s = sqrt(2.0 * airplane.takeoff_weight_lbf / ...
    (constants.RHO_MSL_SLUG_FT3 * airplane.wing_area_ft2 * ...
    airplane.cl_max));

if strcmp(airplane.type, 'propeller')
    performance.power_available_msl_ft_lbf_s = ...
        airplane.propeller_efficiency * airplane.shaft_power_msl_hp ...
        * constants.HP_TO_FT_LBF_S;
else
    performance.power_available_msl_ft_lbf_s = NaN;
end

altitude_grid_ft = analysis.altitude_grid_ft;
num_altitudes = numel(altitude_grid_ft);

density_grid_slug_ft3 = zeros(1, num_altitudes);
minimum_true_speed_ft_s = nan(1, num_altitudes);
maximum_true_speed_ft_s = nan(1, num_altitudes);
stall_true_speed_ft_s = nan(1, num_altitudes);
maximum_roc_ft_min = nan(1, num_altitudes);
maximum_turn_rate_deg_s = nan(1, num_altitudes);
maximum_range_nm = nan(1, num_altitudes);
maximum_endurance_hr = nan(1, num_altitudes);

for altitude_ind = 1:num_altitudes
    altitude_ft = altitude_grid_ft(altitude_ind);

    point = evaluate_altitude_point( ...
        airplane, performance, altitude_ft, constants, analysis);

    density_grid_slug_ft3(altitude_ind) = point.density_slug_ft3;
    minimum_true_speed_ft_s(altitude_ind) = point.minimum_true_speed_ft_s;
    maximum_true_speed_ft_s(altitude_ind) = point.maximum_true_speed_ft_s;
    stall_true_speed_ft_s(altitude_ind) = point.stall_true_speed_ft_s;
```

```
    maximum_roc_ft_min(altitude_ind) = point.maximum_roc_ft_min;
    maximum_turn_rate_deg_s(altitude_ind) = point.maximum_turn_rate_deg_s;
    maximum_range_nm(altitude_ind) = compute_cruise_metric( ...
        airplane, performance, point.density_slug_ft3, constants, ...
        analysis, 'range');
    maximum_endurance_hr(altitude_ind) = compute_cruise_metric( ...
        airplane, performance, point.density_slug_ft3, constants, ...
        analysis, 'endurance');
end

performance.altitude_grid_ft = altitude_grid_ft;
performance.density_grid_slug_ft3 = density_grid_slug_ft3;
performance.minimum_true_speed_ft_s = minimum_true_speed_ft_s;
performance.maximum_true_speed_ft_s = maximum_true_speed_ft_s;
performance.stall_true_speed_ft_s = stall_true_speed_ft_s;
performance.maximum_roc_ft_min = maximum_roc_ft_min;
performance.maximum_turn_rate_deg_s = maximum_turn_rate_deg_s;
performance.maximum_range_nm = maximum_range_nm;
performance.maximum_endurance_hr = maximum_endurance_hr;

performance.service_ceiling_ft = find_ceiling( ...
    altitude_grid_ft, maximum_roc_ft_min, ...
    airplane.service_ceiling_roc_ft_min);
performance.absolute_ceiling_ft = find_ceiling( ...
    altitude_grid_ft, maximum_roc_ft_min, 0.0);
performance.half_service_ceiling_ft = 0.5 * performance.service_ceiling_ft;

performance.plot_altitudes_ft = [ ...
    0.0, ...
    performance.half_service_ceiling_ft, ...
    performance.service_ceiling_ft];

for point_ind = 1:3
    altitude_ft = performance.plot_altitudes_ft(point_ind);

    performance.plot_points(point_ind) = ...
        evaluate_altitude_point(airplane, performance, ...
        altitude_ft, constants, analysis);
end

absolute_mask = altitude_grid_ft <= performance.absolute_ceiling_ft;

finite_speed_mask = absolute_mask & isfinite(maximum_true_speed_ft_s);

performance.overall_max_true_speed_ft_s = max( ...
    maximum_true_speed_ft_s(finite_speed_mask));
end

% This function evaluates the performance quantities at a single altitude.
function point = evaluate_altitude_point(airplane, performance, ...
    altitude_ft, constants, analysis)

point = struct();
```

```
point.altitude_ft = altitude_ft;
point.density_slug_ft3 = standard_atmosphere_density(altitude_ft);
point.density_ratio = point.density_slug_ft3 / ...
    constants.RHO_MSL_SLUG_FT3;
point.true_speed_grid_ft_s = analysis.equivalent_speed_grid_ft_s ./ ...
    sqrt(point.density_ratio);

point.drag_required_lbf = drag_required_from_eas( ...
    analysis.equivalent_speed_grid_ft_s, airplane, performance, ...
    constants);

if strcmp(airplane.type, 'jet')
    point.thrust_available_lbf = point.density_ratio * ...
        airplane.thrust_available_msl_lbf .* ...
        ones(size(analysis.equivalent_speed_grid_ft_s));
    point.power_available_ft_lbf_s = point.thrust_available_lbf .* ...
        point.true_speed_grid_ft_s;
else
    point.power_available_ft_lbf_s = point.density_ratio * ...
        performance.power_available_msl_ft_lbf_s .* ...
        ones(size(analysis.equivalent_speed_grid_ft_s));
    point.thrust_available_lbf = point.power_available_ft_lbf_s ./ ...
        point.true_speed_grid_ft_s;
end

point.power_required_ft_lbf_s = point.drag_required_lbf .* ...
    point.true_speed_grid_ft_s;

if strcmp(airplane.type, 'jet')
    level_excess = point.thrust_available_lbf - point.drag_required_lbf;
    excess_power_ft_lbf_s = level_excess .* point.true_speed_grid_ft_s;
else
    level_excess = point.power_available_ft_lbf_s - ...
        point.power_required_ft_lbf_s;
    excess_power_ft_lbf_s = level_excess;
end

[min_eas_ft_s, max_eas_ft_s] = find_level_speed_bounds( ...
    analysis.equivalent_speed_grid_ft_s, level_excess, ...
    performance.stall_eas_ft_s);

point.minimum_eas_ft_s = min_eas_ft_s;
point.maximum_eas_ft_s = max_eas_ft_s;
point.stall_eas_ft_s = performance.stall_eas_ft_s;
point.stall_true_speed_ft_s = performance.stall_eas_ft_s / ...
    sqrt(point.density_ratio);

point.minimum_true_speed_ft_s = min_eas_ft_s / ...
    sqrt(point.density_ratio);
point.maximum_true_speed_ft_s = max_eas_ft_s / ...
    sqrt(point.density_ratio);

point.maximum_roc_ft_min = constants.MIN_PER_HR * ...
    max(excess_power_ft_lbf_s) / airplane.takeoff_weight_lbf;
```

```
point.maximum_turn_rate_deg_s = compute_max_turn_rate( ...
    airplane, performance, point, constants);

point.range_eas_ft_s = clamp_speed_to_bounds( ...
    theoretical_optimum_eas(airplane, performance, constants, 'range'), ...
    min_eas_ft_s, max_eas_ft_s);
point.endurance_eas_ft_s = clamp_speed_to_bounds( ...
    theoretical_optimum_eas(airplane, performance, constants, ...
    'endurance'), min_eas_ft_s, max_eas_ft_s);

point.minimum_eas_kt = point.minimum_eas_ft_s * constants.FT_S_TO_KT;
point.maximum_eas_kt = point.maximum_eas_ft_s * constants.FT_S_TO_KT;
point.stall_eas_kt = point.stall_eas_ft_s * constants.FT_S_TO_KT;
point.range_eas_kt = point.range_eas_ft_s * constants.FT_S_TO_KT;
point.endurance_eas_kt = point.endurance_eas_ft_s * constants.FT_S_TO_KT;

point.minimum_true_speed_kt = point.minimum_true_speed_ft_s * ...
    constants.FT_S_TO_KT;
point.maximum_true_speed_kt = point.maximum_true_speed_ft_s * ...
    constants.FT_S_TO_KT;
point.stall_true_speed_kt = point.stall_true_speed_ft_s * ...
    constants.FT_S_TO_KT;

point.power_available_hp = point.power_available_ft_lbf_s / ...
    constants.HP_TO_FT_LBF_S;
point.power_required_hp = point.power_required_ft_lbf_s / ...
    constants.HP_TO_FT_LBF_S;
end

% This function evaluates the standard atmosphere density.
function density_slug_ft3 = standard_atmosphere_density(altitude_ft)

if altitude_ft < 36090.0
    density_slug_ft3 = (6.6277e-15) * ...
        (518.69 - 0.0035662 * altitude_ft)^4.256;
else
    density_slug_ft3 = 0.0030638 * exp((-4.8063e-5) * altitude_ft);
end
end

% This function computes the drag required from equivalent airspeed.
function drag_required_lbf = ...
    drag_required_from_eas(equivalent_speed_ft_s, airplane, ...
    performance, constants)

q_msl_lbf_ft2 = 0.5 * constants.RHO_MSL_SLUG_FT3 .* ...
    equivalent_speed_ft_s.^2;

drag_required_lbf = q_msl_lbf_ft2 * airplane.wing_area_ft2 ...
    * airplane.cd0 + performance.induced_drag_factor ...
    * airplane.takeoff_weight_lbf^2 ./ ...
    (q_msl_lbf_ft2 * airplane.wing_area_ft2);
end
```

```
% This function determines the minimum and maximum level-flight equivalent
% airspeeds.
function [minimum_eas_ft_s, maximum_eas_ft_s] = ...
    find_level_speed_bounds(equivalent_speed_grid_ft_s, level_excess, ...
    stall_eas_ft_s)

feasible_ind = find(level_excess >= 0.0);

if isempty(feasible_ind)
    minimum_eas_ft_s = NaN;
    maximum_eas_ft_s = NaN;
    return;
end

first_ind = feasible_ind(1);
last_ind = feasible_ind(end);

minimum_eas_ft_s = interpolate_boundary_speed( ...
    equivalent_speed_grid_ft_s, level_excess, first_ind, 'lower');
maximum_eas_ft_s = interpolate_boundary_speed( ...
    equivalent_speed_grid_ft_s, level_excess, last_ind, 'upper');

minimum_eas_ft_s = max(minimum_eas_ft_s, stall_eas_ft_s);

if minimum_eas_ft_s > maximum_eas_ft_s
    minimum_eas_ft_s = NaN;
    maximum_eas_ft_s = NaN;
end
end

% This function interpolates the level-flight boundary speed.
function boundary_speed_ft_s = interpolate_boundary_speed( ...
    equivalent_speed_grid_ft_s, level_excess, boundary_ind, side)

boundary_speed_ft_s = equivalent_speed_grid_ft_s(boundary_ind);

if strcmp(side, 'lower')
    if boundary_ind == 1
        return;
    end

    left_ind = boundary_ind - 1;
    right_ind = boundary_ind;
else
    if boundary_ind == numel(level_excess)
        return;
    end

    left_ind = boundary_ind;
    right_ind = boundary_ind + 1;
end

if level_excess(left_ind) == level_excess(right_ind)
    boundary_speed_ft_s = equivalent_speed_grid_ft_s(boundary_ind);
```

```
    return;
end

boundary_speed_ft_s = equivalent_speed_grid_ft_s(left_ind) + ...
    (0.0 - level_excess(left_ind)) * ...
    (equivalent_speed_grid_ft_s(right_ind) ...
    - equivalent_speed_grid_ft_s(left_ind)) / ...
    (level_excess(right_ind) - level_excess(left_ind));
end

% This function computes the maximum sustained turn rate.
function turn_rate_deg_s = compute_max_turn_rate(airplane, performance, ...
    point, constants)

true_speed_grid_ft_s = point.true_speed_grid_ft_s;

dynamic_pressure_lbf_ft2 = 0.5 * point.density_slug_ft3 .* ...
    true_speed_grid_ft_s.^2;

stall_load_factor = dynamic_pressure_lbf_ft2 * airplane.wing_area_ft2 ...
    * airplane.cl_max / airplane.takeoff_weight_lbf;

if strcmp(airplane.type, 'jet')
    effective_thrust_lbf = point.thrust_available_lbf;
else
    effective_thrust_lbf = point.power_available_ft_lbf_s ./ ...
        true_speed_grid_ft_s;
end

propulsion_term = (effective_thrust_lbf - dynamic_pressure_lbf_ft2 ...
    * airplane.wing_area_ft2 * airplane.cd0) .* ...
    dynamic_pressure_lbf_ft2 * airplane.wing_area_ft2 ./ ...
    (performance.induced_drag_factor * airplane.takeoff_weight_lbf^2);
propulsion_term = max(propulsion_term, 0.0);

propulsion_load_factor = sqrt(propulsion_term);

max_sustained_load_factor = min([ ...
    stall_load_factor; ...
    propulsion_load_factor; ...
    airplane.n_max * ones(size(stall_load_factor))], [], 1);

turn_rate_rad_s = constants.G0_FT_S2 .* sqrt(max( ...
    max_sustained_load_factor.^2 - 1.0, 0.0)) ./ true_speed_grid_ft_s;
turn_rate_rad_s(max_sustained_load_factor < 1.0) = NaN;

turn_rate_deg_s = max(turn_rate_rad_s) * constants.RAD_TO_DEG;
end

% This function computes the theoretical optimum equivalent airspeed.
function optimum_eas_ft_s = theoretical_optimum_eas(airplane, ...
    performance, constants, mode)

if strcmp(airplane.type, 'jet')
```

```
    if strcmp(mode, 'range')
        optimum_cl = sqrt(airplane.cd0 / ...
            (3.0 * performance.induced_drag_factor));
    else
        optimum_cl = sqrt(airplane.cd0 / ...
            performance.induced_drag_factor);
    end
else
    if strcmp(mode, 'range')
        optimum_cl = sqrt(airplane.cd0 / ...
            performance.induced_drag_factor);
    else
        optimum_cl = sqrt(3.0 * airplane.cd0 / ...
            performance.induced_drag_factor);
    end
end

optimum_eas_ft_s = sqrt(2.0 * airplane.takeoff_weight_lbf / ...
    (constants.RHO_MSL_SLUG_FT3 * airplane.wing_area_ft2 * ...
    optimum_cl));
end

% This function limits the speed to the feasible bounds.
function clamped_speed_ft_s = clamp_speed_to_bounds(speed_ft_s, ...
    minimum_speed_ft_s, maximum_speed_ft_s)

if isnan(minimum_speed_ft_s) || isnan(maximum_speed_ft_s)
    clamped_speed_ft_s = NaN;
    return;
end

clamped_speed_ft_s = min(max(speed_ft_s, minimum_speed_ft_s), ...
    maximum_speed_ft_s);
end

% This function computes the cruise range or endurance metric.
function cruise_metric = compute_cruise_metric(airplane, performance, ...
    density_slug_ft3, constants, analysis, mode)

initial_weight_lbf = airplane.takeoff_weight_lbf;
final_weight_lbf = airplane.takeoff_weight_lbf - airplane.fuel_weight_lbf;

weight_grid_lbf = linspace(initial_weight_lbf, final_weight_lbf, ...
    analysis.weight_num_steps);
weight_step_lbf = -diff(weight_grid_lbf);
mid_weight_lbf = 0.5 * (weight_grid_lbf(1:end - 1) ...
    + weight_grid_lbf(2:end));

if strcmp(airplane.type, 'jet')
    if strcmp(mode, 'range')
        optimum_cl = sqrt(airplane.cd0 / ...
            (3.0 * performance.induced_drag_factor));
    else
        optimum_cl = sqrt(airplane.cd0 / ...
```

```
            performance.induced_drag_factor);
    end
else
    if strcmp(mode, 'range')
        optimum_cl = sqrt(airplane.cd0 / ...
            performance.induced_drag_factor);
    else
        optimum_cl = sqrt(3.0 * airplane.cd0 / ...
            performance.induced_drag_factor);
    end
end

optimum_cd = airplane.cd0 + performance.induced_drag_factor ...
    * optimum_cl^2;

speed_ft_s = sqrt(2.0 * mid_weight_lbf ./ ...
    (density_slug_ft3 * airplane.wing_area_ft2 * optimum_cl));
drag_lbf = mid_weight_lbf * optimum_cd / optimum_cl;

if strcmp(airplane.type, 'jet')
    fuel_burn_lbf_hr = airplane.fuel_rate .* drag_lbf;
else
    shaft_power_required_hp = drag_lbf .* speed_ft_s ./ ...
        (airplane.propeller_efficiency * constants.HP_TO_FT_LBF_S);
    fuel_burn_lbf_hr = airplane.fuel_rate .* shaft_power_required_hp;
end

time_step_hr = weight_step_lbf ./ fuel_burn_lbf_hr;

if strcmp(mode, 'range')
    cruise_metric = sum(speed_ft_s .* constants.SEC_PER_HR .* ...
        time_step_hr) * constants.FT_TO_NM;
else
    cruise_metric = sum(time_step_hr);
end
end

% This function interpolates the altitude of the target rate of climb.
function ceiling_ft = find_ceiling(altitude_grid_ft, roc_grid_ft_min, ...
    target_roc_ft_min)

crossing_ind = find(roc_grid_ft_min < target_roc_ft_min, 1, 'first');

if isempty(crossing_ind)
    ceiling_ft = altitude_grid_ft(end);
    return;
end

if crossing_ind == 1
    ceiling_ft = altitude_grid_ft(1);
    return;
end

left_ind = crossing_ind - 1;
```

```
right_ind = crossing_ind;

ceiling_ft = altitude_grid_ft(left_ind) + ...
    (target_roc_ft_min - roc_grid_ft_min(left_ind)) * ...
    (altitude_grid_ft(right_ind) - altitude_grid_ft(left_ind)) / ...
    (roc_grid_ft_min(right_ind) - roc_grid_ft_min(left_ind));
end

% This function prints the summary performance results.
function print_summary(performance, constants)

fprintf('\n%s\n', upper(performance.airplane.name));
fprintf('Service ceiling: %.0f ft\n', performance.service_ceiling_ft);
fprintf('Absolute ceiling: %.0f ft\n', performance.absolute_ceiling_ft);
fprintf('Stall EAS: %.2f kt\n', performance.stall_eas_ft_s ...
    * constants.FT_S_TO_KT);
fprintf('Overall maximum true airspeed: %.2f kt\n', ...
    performance.overall_max_true_speed_ft_s * constants.FT_S_TO_KT);
end

% This function creates the required project plots.
function make_required_plots(performance, constants, analysis)

plot_thrust_and_drag(performance, constants, analysis);
plot_altitude_vs_true_speed(performance, constants, analysis);
plot_power_curves(performance, constants, analysis);
plot_maximum_roc(performance, analysis);
plot_maximum_turn_rate(performance, analysis);
plot_vn_diagram(performance, constants, analysis, 1);
plot_vn_diagram(performance, constants, analysis, 2);
plot_vn_diagram(performance, constants, analysis, 3);
plot_maximum_range(performance, analysis);
plot_maximum_endurance(performance, analysis);
end

% This function creates the thrust and drag plot.
function plot_thrust_and_drag(performance, constants, analysis)

figure('Name', [performance.airplane.name, ' Plot 1']);
hold on;
grid on;
box on;

for point_ind = 1:numel(performance.plot_points)
    plot(analysis.equivalent_speed_grid_ft_s * constants.FT_S_TO_KT, ...
        performance.plot_points(point_ind).thrust_available_lbf, '-', ...
        'Color', analysis.curve_colors(point_ind, :), 'LineWidth', 1.6, ...
        'DisplayName', [analysis.point_altitude_labels{point_ind}, ...
        ' thrust']);
end

plot(analysis.equivalent_speed_grid_ft_s * constants.FT_S_TO_KT, ...
    performance.plot_points(1).drag_required_lbf, '-', 'Color', ...
    analysis.drag_color, 'LineWidth', 1.8, 'DisplayName', 'Drag');
```

```
xline(performance.stall_eas_ft_s * constants.FT_S_TO_KT, '--', ...
    'Color', analysis.stall_color, 'LineWidth', 1.3, 'DisplayName', ...
    'Stall EAS');

for point_ind = 1:numel(performance.plot_points)
    label = analysis.point_altitude_labels{point_ind};
    point = performance.plot_points(point_ind);

    add_speed_marker(point.maximum_eas_kt, drag_value_from_speed(point, ...
    point.maximum_eas_ft_s), ['V_{max}, ', label], 'o', ...
    analysis.curve_colors(point_ind, :), analysis.marker_size, ...
    6.0, 80.0, 'left', 'bottom');

add_speed_marker(point.minimum_eas_kt, drag_value_from_speed(point, ...
    point.minimum_eas_ft_s), ['V_{min}, ', label], 's', ...
    analysis.curve_colors(point_ind, :), analysis.marker_size, ...
    6.0, -80.0, 'left', 'top');

if strcmp(performance.airplane.type, 'jet')
    add_speed_marker(point.range_eas_kt, drag_value_from_speed( ...
        point, point.range_eas_ft_s), ['V_{R}, ', label], 'd', ...
        analysis.curve_colors(point_ind, :), analysis.marker_size, ...
        -8.0, 70.0, 'right', 'bottom');

    add_speed_marker(point.endurance_eas_kt, drag_value_from_speed( ...
        point, point.endurance_eas_ft_s), ['V_{E}, ', label], '^', ...
        analysis.curve_colors(point_ind, :), analysis.marker_size, ...
        -8.0, -70.0, 'right', 'top');
end
end

display_min_speed_kt = 0.85 * performance.plot_points(1).stall_eas_kt;
display_max_speed_kt = 1.05 * ...
    max([performance.plot_points.maximum_eas_kt]);
xlim([display_min_speed_kt, display_max_speed_kt]);

xlabel('Sea level equivalent airspeed, V_{EAS} (kt)');
ylabel('Thrust or drag (lbf)');
title([performance.airplane.name, ': thrust and drag vs. EAS']);
legend('Location', 'best');
set(gca, 'FontSize', analysis.figure_font_size);
end

% This function plots altitude versus minimum and maximum airspeed.
function plot_altitude_vs_true_speed(performance, constants, analysis)

figure('Name', [performance.airplane.name, ' Plot 2']);
hold on;
grid on;
box on;

altitude_plot_ft = performance.altitude_grid_ft( ...
    performance.altitude_grid_ft <= performance.absolute_ceiling_ft);
```

```
plot(performance.minimum_true_speed_ft_s(1:numel(altitude_plot_ft)) ...
    * constants.FT_S_TO_KT, altitude_plot_ft, '-', 'LineWidth', 1.7, ...
    'DisplayName', 'Minimum true airspeed');
plot(performance.maximum_true_speed_ft_s(1:numel(altitude_plot_ft)) ...
    * constants.FT_S_TO_KT, altitude_plot_ft, '-', 'LineWidth', 1.7, ...
    'DisplayName', 'Maximum true airspeed');
plot(performance.stall_true_speed_ft_s(1:numel(altitude_plot_ft)) ...
    * constants.FT_S_TO_KT, altitude_plot_ft, '--', 'LineWidth', 1.4, ...
    'DisplayName', 'Stall airspeed');

xline(performance.overall_max_true_speed_ft_s * constants.FT_S_TO_KT, ...
    '--', 'LineWidth', 1.3, 'DisplayName', 'Overall maximum airspeed');

xlabel('True airspeed, V_{TAS} (kt)');
ylabel('Altitude above MSL (ft)');
title([performance.airplane.name, ': altitude vs. min and max TAS']);
legend('Location', 'best');
set(gca, 'FontSize', analysis.figure_font_size);
end

% This function plots power required and power available versus equivalent
% airspeed.
function plot_power_curves(performance, constants, analysis)

figure('Name', [performance.airplane.name, ' Plot 3']);
hold on;
grid on;
box on;

for point_ind = 1:numel(performance.plot_points)
    point = performance.plot_points(point_ind);

    plot(analysis.equivalent_speed_grid_ft_s * constants.FT_S_TO_KT, ...
        point.power_required_hp, '-', 'Color', ...
        analysis.curve_colors(point_ind, :), 'LineWidth', 1.6, ...
        'DisplayName', [analysis.point_altitude_labels{point_ind}, ...
        ' power required']);
    plot(analysis.equivalent_speed_grid_ft_s * constants.FT_S_TO_KT, ...
        point.power_available_hp, '--', 'Color', ...
        analysis.curve_colors(point_ind, :), 'LineWidth', 1.6, ...
        'DisplayName', [analysis.point_altitude_labels{point_ind}, ...
        ' power available']);
end

xline(performance.stall_eas_ft_s * constants.FT_S_TO_KT, '--', ...
    'Color', analysis.stall_color, 'LineWidth', 1.3, 'DisplayName', ...
    'Stall EAS');

for point_ind = 1:numel(performance.plot_points)
    label = analysis.point_altitude_labels{point_ind};
    point = performance.plot_points(point_ind);

    add_speed_marker(point.maximum_eas_kt, power_value_from_speed( ...
```

```
    point, point.maximum_eas_ft_s, constants), ['V_{max}, ', label], ...
    'o', analysis.curve_colors(point_ind, :), analysis.marker_size, ...
    6.0, 250.0, 'left', 'bottom');

add_speed_marker(point.minimum_eas_kt, power_value_from_speed( ...
    point, point.minimum_eas_ft_s, constants), ['V_{min}, ', label], ...
    's', analysis.curve_colors(point_ind, :), analysis.marker_size, ...
    6.0, -250.0, 'left', 'top');

if strcmp(performance.airplane.type, 'propeller')
    add_speed_marker(point.range_eas_kt, power_value_from_speed( ...
        point, point.range_eas_ft_s, constants), ['V_{R}, ', label], ...
        'd', analysis.curve_colors(point_ind, :), analysis.marker_size, ...
        -8.0, 220.0, 'right', 'bottom');

    add_speed_marker(point.endurance_eas_kt, power_value_from_speed( ...
        point, point.endurance_eas_ft_s, constants), ...
        ['V_{E}, ', label], '^', analysis.curve_colors(point_ind, :), ...
        analysis.marker_size, -8.0, -220.0, 'right', 'top');
end
end

display_min_speed_kt = 0.85 * performance.plot_points(1).stall_eas_kt;
display_max_speed_kt = 1.05 * ...
    max([performance.plot_points.maximum_eas_kt]);
xlim([display_min_speed_kt, display_max_speed_kt]);

xlabel('Sea level equivalent airspeed, V_{EAS} (kt)');
ylabel('Power (hp)');
title([performance.airplane.name, ': power required and available']);
legend('Location', 'best');
set(gca, 'FontSize', analysis.figure_font_size);
end

% This function plots the maximum rate of climb versus altitude.
function plot_maximum_roc(performance, analysis)

figure('Name', [performance.airplane.name, ' Plot 4']);
hold on;
grid on;
box on;

service_mask = performance.altitude_grid_ft <= ...
    performance.service_ceiling_ft;

plot(performance.altitude_grid_ft(service_mask), ...
    performance.maximum_roc_ft_min(service_mask), '-', 'LineWidth', 1.8);

xlabel('Altitude above MSL (ft)');
ylabel('Maximum rate of climb (ft/min)');
title([performance.airplane.name, ': maximum rate of climb']);
set(gca, 'FontSize', analysis.figure_font_size);
end
```

```
% This function plots the maximum turn rate versus altitude.
function plot_maximum_turn_rate(performance, analysis)

figure('Name', [performance.airplane.name, ' Plot 5']);
hold on;
grid on;
box on;

service_mask = performance.altitude_grid_ft <= ...
    performance.service_ceiling_ft;

plot(performance.altitude_grid_ft(service_mask), ...
    performance.maximum_turn_rate_deg_s(service_mask), '-', ...
    'LineWidth', 1.8);

xlabel('Altitude above MSL (ft)');
ylabel('Maximum turn rate (deg/s)');
title([performance.airplane.name, ': maximum turn rate']);
set(gca, 'FontSize', analysis.figure_font_size);
end

% This function plots the V-n diagram at a selected altitude.
function plot_vn_diagram(performance, constants, analysis, point_ind)

point = performance.plot_points(point_ind);

figure_number = 5 + point_ind;
figure('Name', [performance.airplane.name, ' Plot ', ...
    num2str(figure_number)]);
hold on;
grid on;
box on;

max_true_speed_ft_s = point.maximum_true_speed_ft_s;
true_speed_grid_ft_s = linspace(0.0, max_true_speed_ft_s, 500);

stall_speed_ft_s = point.stall_true_speed_ft_s;
stall_load_factor = (true_speed_grid_ft_s ./ stall_speed_ft_s).^2;
positive_envelope = min(stall_load_factor, performance.airplane.n_max);
negative_envelope = max(-stall_load_factor, -performance.airplane.n_max);

speed_kt = true_speed_grid_ft_s * constants.FT_S_TO_KT;

plot(speed_kt, positive_envelope, '-', ...
    'LineWidth', 1.8, 'DisplayName', 'Positive envelope');
speed_kt = true_speed_grid_ft_s * constants.FT_S_TO_KT;

plot(speed_kt, negative_envelope, '-', ...
    'LineWidth', 1.8, 'DisplayName', 'Negative envelope');

xline(point.maximum_true_speed_kt, '--', 'LineWidth', 1.3, ...
    'DisplayName', 'Maximum airspeed');
yline(performance.airplane.n_max, '--', 'LineWidth', 1.1, ...
    'DisplayName', 'Positive n_{max}');
```

```
yline(-performance.airplane.n_max, '--', 'LineWidth', 1.1, ...
    'DisplayName', 'Negative n_{max}');

xlabel('True airspeed, V_{TAS} (kt)');
ylabel('Load factor, n');
title([performance.airplane.name, ': V-n diagram at ', ...
    analysis.point_altitude_labels{point_ind}]);

xlim([0.0, point.maximum_true_speed_kt]);
ylim([-performance.airplane.n_max, performance.airplane.n_max]);

legend('Location', 'best');
set(gca, 'FontSize', analysis.figure_font_size);
end

% This function plots the maximum range versus altitude.
function plot_maximum_range(performance, analysis)
    figure('Name', [performance.airplane.name, ' Plot 9']);
    hold on;
    grid on;
    box on;

    service_mask = performance.altitude_grid_ft <= ...
        performance.service_ceiling_ft;

    range_nm = performance.maximum_range_nm(service_mask);
    altitude_ft = performance.altitude_grid_ft(service_mask);

    plot(altitude_ft, range_nm, '-', 'LineWidth', 1.8);

    xlim([0.0, performance.service_ceiling_ft]);

    if max(range_nm) - min(range_nm) < 5.0
        range_mid = mean(range_nm);
        ylim([0.95, 1.05] * range_mid);
        ytickformat('%.0f');
    end

    xlabel('Altitude above MSL (ft)');
    ylabel('Maximum range (nmi)');
    title([performance.airplane.name, ': maximum range']);
    set(gca, 'FontSize', analysis.figure_font_size);
end

% This function plots the maximum endurance versus altitude.
function plot_maximum_endurance(performance, analysis)

figure('Name', [performance.airplane.name, ' Plot 10']);
hold on;
grid on;
box on;

service_mask = performance.altitude_grid_ft <= ...
    performance.service_ceiling_ft;
```

```
plot(performance.altitude_grid_ft(service_mask), ...
    performance.maximum_endurance_hr(service_mask), '-', ...
    'LineWidth', 1.8);

xlabel('Altitude above MSL (ft)');
ylabel('Maximum endurance (hr)');
title([performance.airplane.name, ': maximum endurance']);
set(gca, 'FontSize', analysis.figure_font_size);
end

% This function interpolates the drag value from the speed.
function drag_value_lbf = drag_value_from_speed(point, speed_ft_s)

if isnan(speed_ft_s)
    drag_value_lbf = NaN;
    return;
end

drag_value_lbf = interp1(point.true_speed_grid_ft_s .* sqrt( ...
    point.density_ratio), point.drag_required_lbf, speed_ft_s, 'linear');
end

% This function interpolates the power value at a selected speed.
function power_value_hp = power_value_from_speed(point, speed_ft_s, ...
    ~)

if isnan(speed_ft_s)
    power_value_hp = NaN;
    return;
end

power_value_hp = interp1(point.true_speed_grid_ft_s .* sqrt( ...
    point.density_ratio), point.power_required_hp, speed_ft_s, 'linear');
end

% This function adds a speed marker and label to a plot.
function add_speed_marker(speed_kt, value_y, label_text, marker_style, ...
    marker_color, marker_size, dx_kt, dy_value, horizontal_align, ...
    vertical_align)

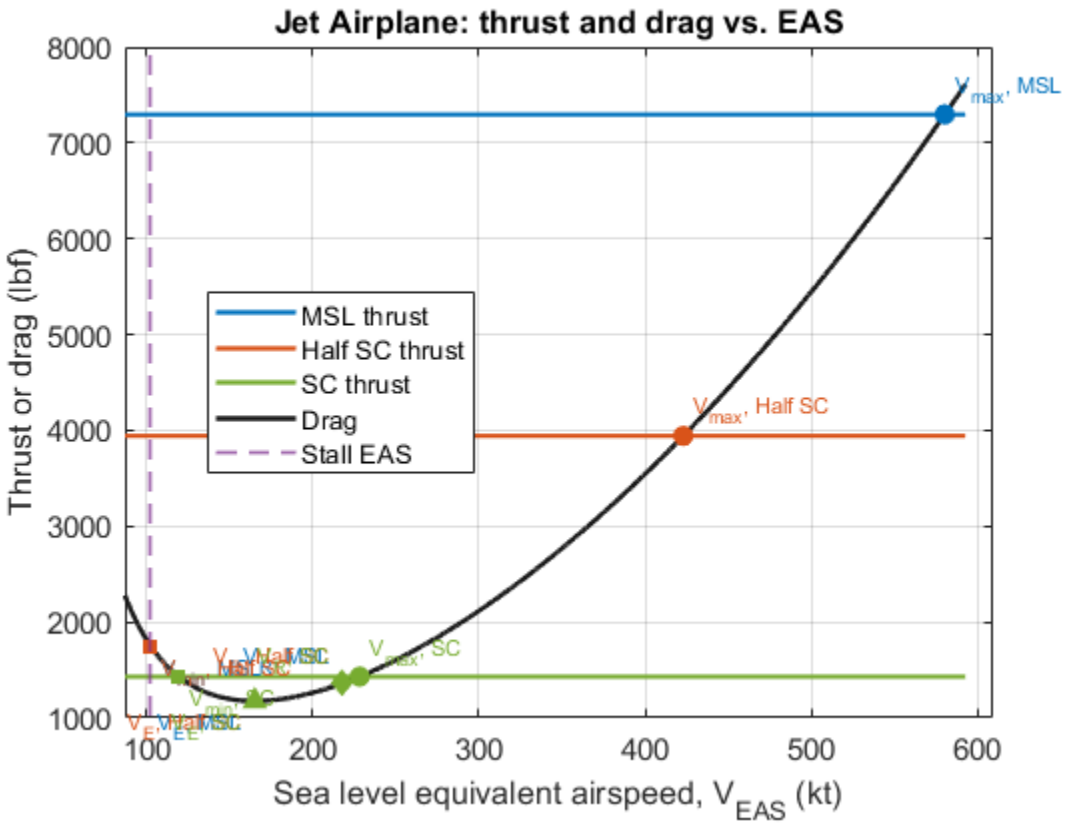
if isnan(speed_kt) || isnan(value_y)
    return;
end

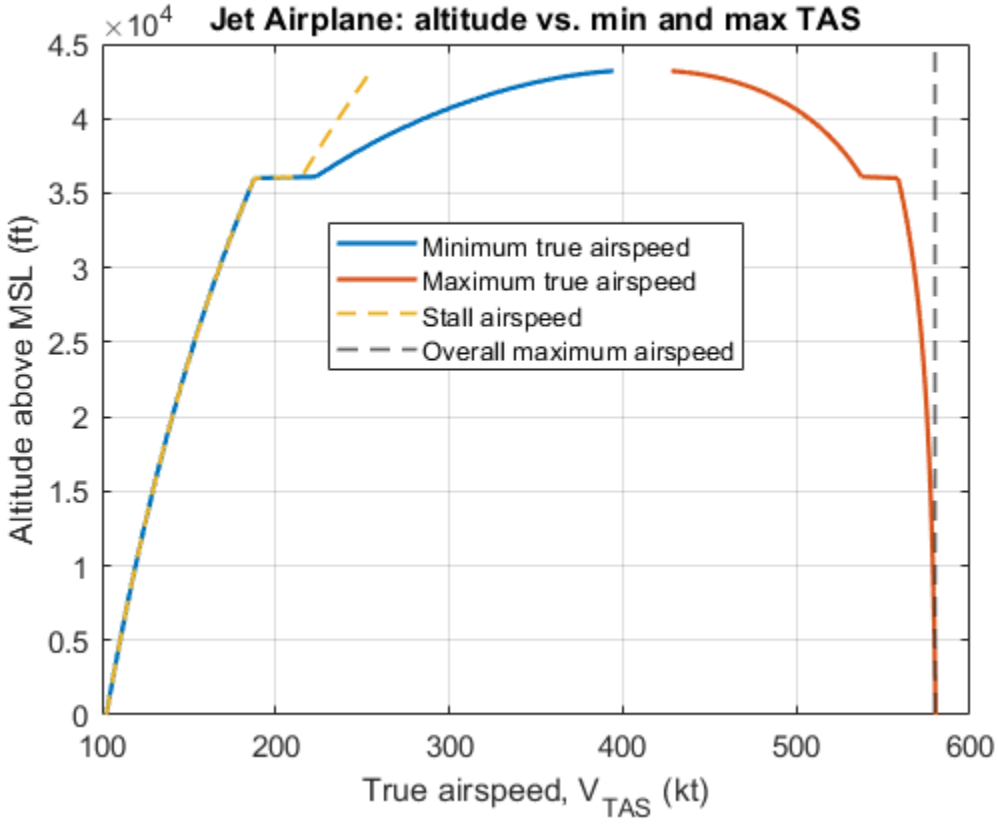
plot(speed_kt, value_y, marker_style, 'MarkerSize', marker_size, ...
    'MarkerFaceColor', marker_color, 'MarkerEdgeColor', marker_color, ...
    'HandleVisibility', 'off');

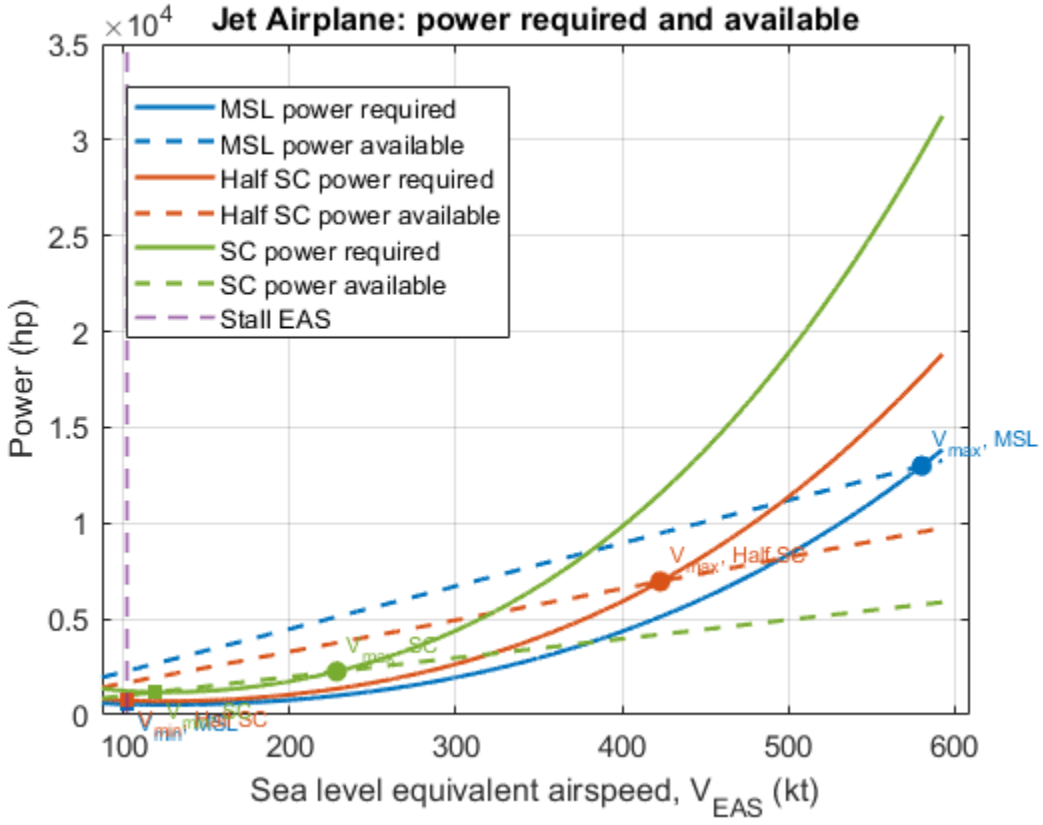
text(speed_kt + dx_kt, value_y + dy_value, label_text, ...
    'Color', marker_color, 'FontSize', 8, ...
    'HorizontalAlignment', horizontal_align, ...
    'VerticalAlignment', vertical_align, ...
    'Interpreter', 'tex', 'HandleVisibility', 'off');
end
```

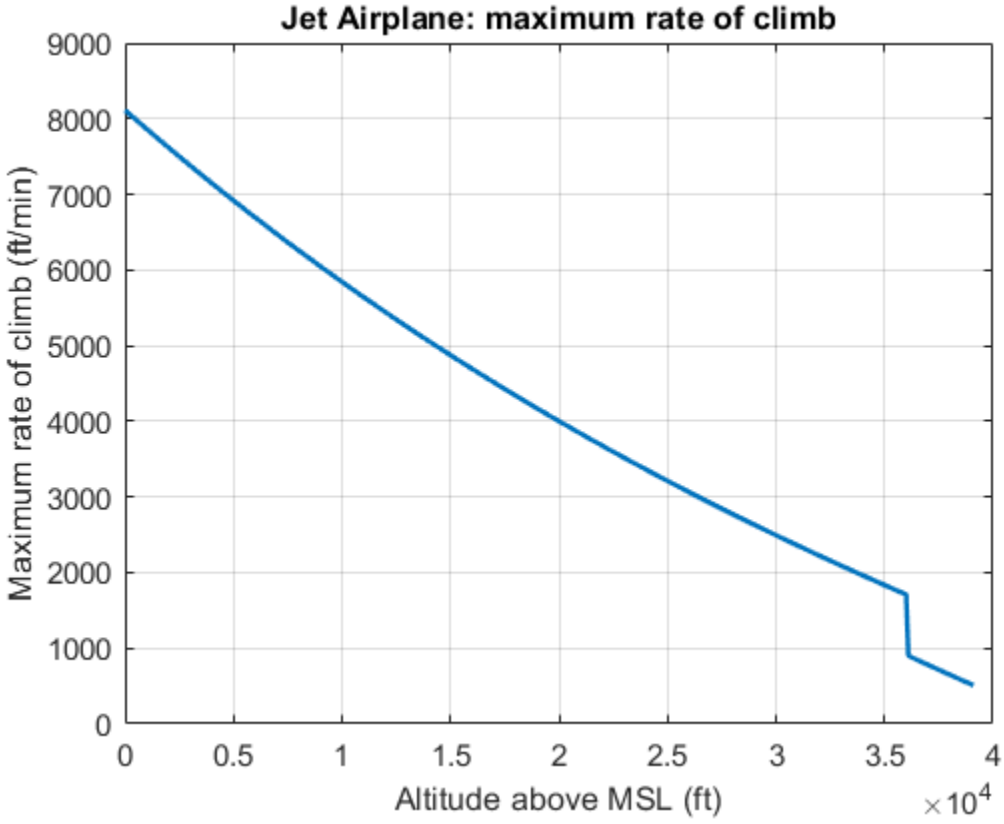
JET AIRPLANE
Service ceiling: 39179 ft
Absolute ceiling: 43268 ft
Stall EAS: 102.53 kt
Overall maximum true airspeed: 580.22 kt

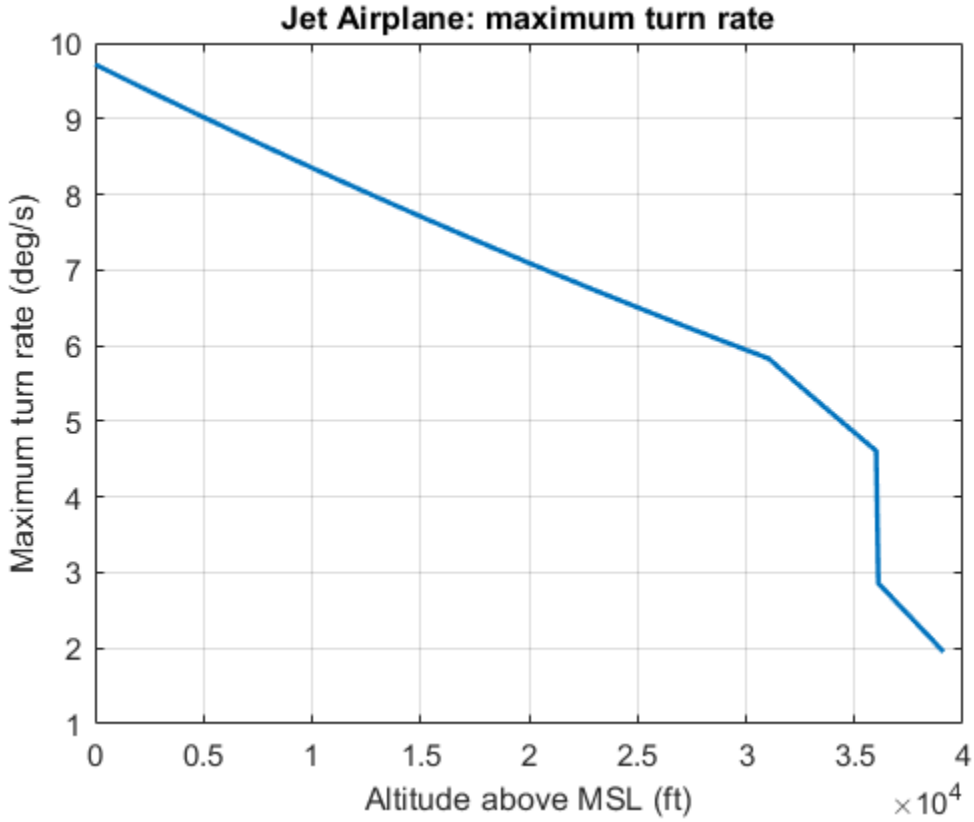
PROPELLER AIRPLANE
Service ceiling: 24683 ft
Absolute ceiling: 26810 ft
Stall EAS: 50.03 kt
Overall maximum true airspeed: 154.94 kt

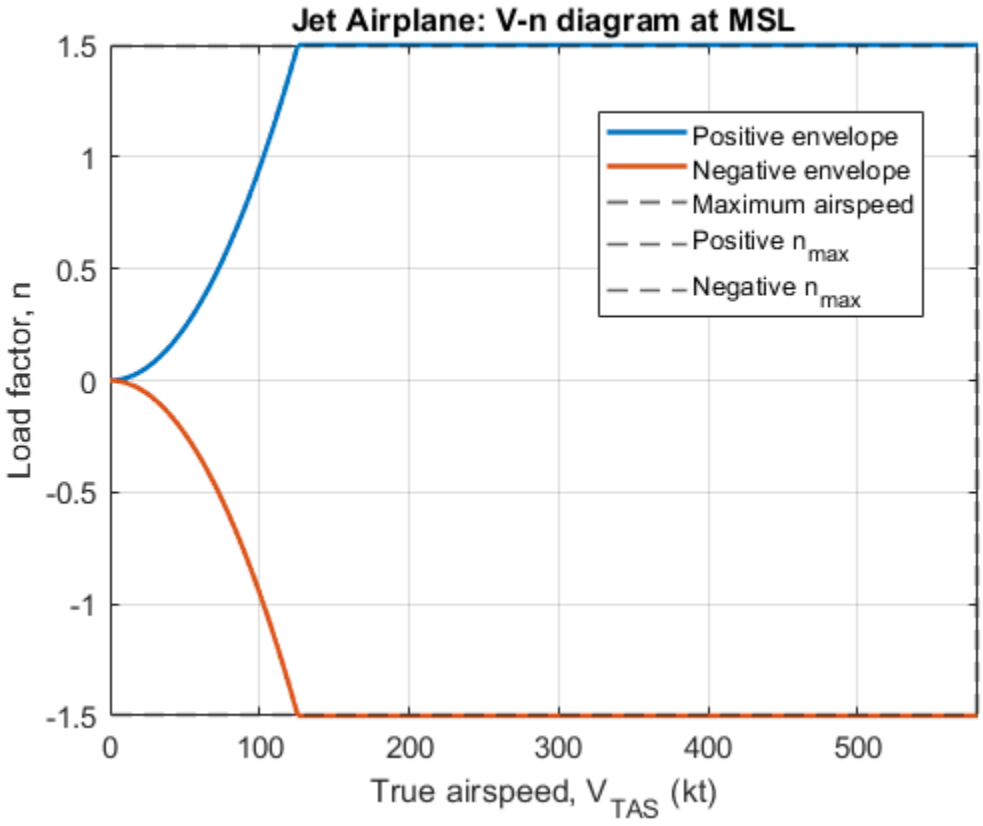


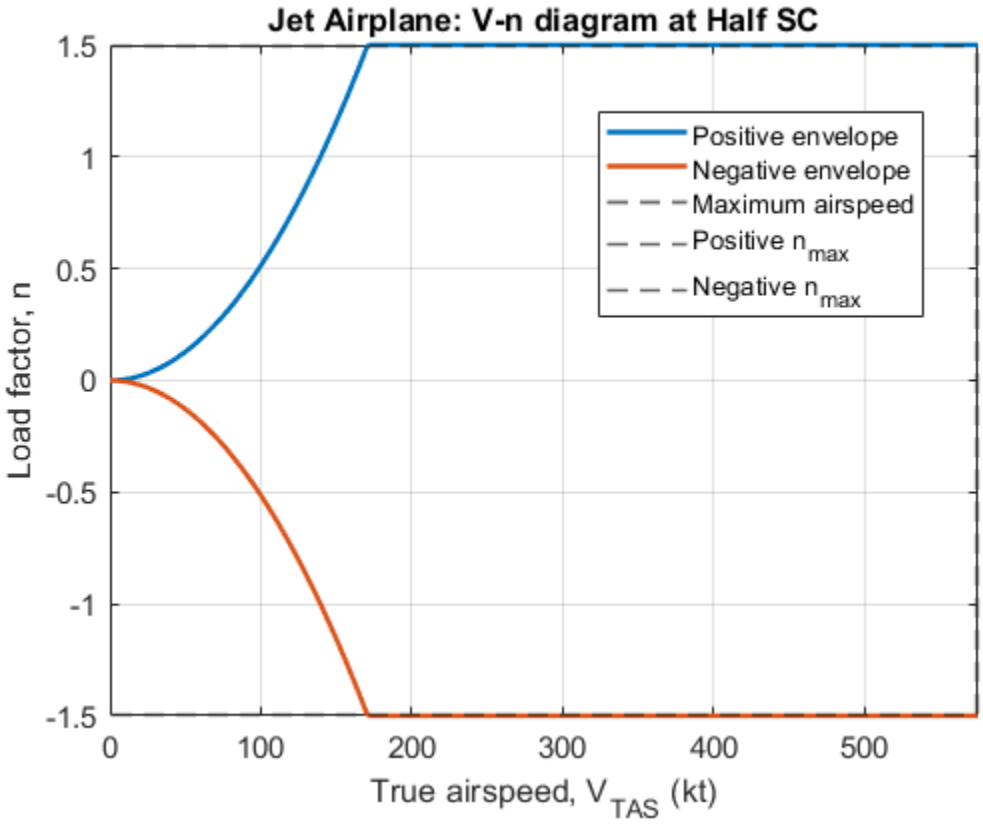

Jet Airplane: altitude vs. min and max TAS
×10^4
4.5
4
3.5
3
2.5
2
1.5
1
0.5
0
Altitude above MSL (ft)
100
200
300
400
500
600
True airspeed, V_TAS (kt)
Minimum true airspeed
Maximum true airspeed
Stall airspeed
Overall maximum airspeed

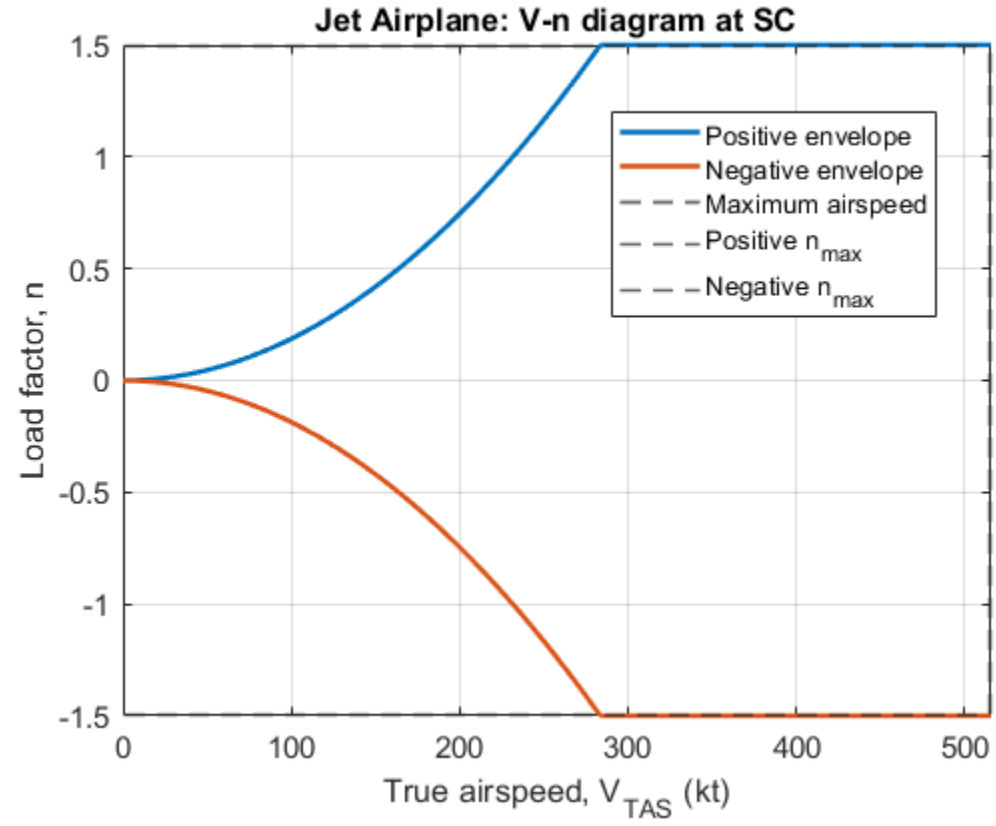

Jet Airplane: power required and available
×10^4
3.5
3
2.5
2
1.5
1
0.5
0
Power (hp)
100
200
300
400
500
600
Sea level equivalent airspeed, V_EAS (kt)
MSL power required
MSL power available
Half SC power required
Half SC power available
SC power required
SC power available
Stall EAS
V max, MSL
V max, Half SC
V max, SC
V min, MSL
V min, Half SC
V min, SC

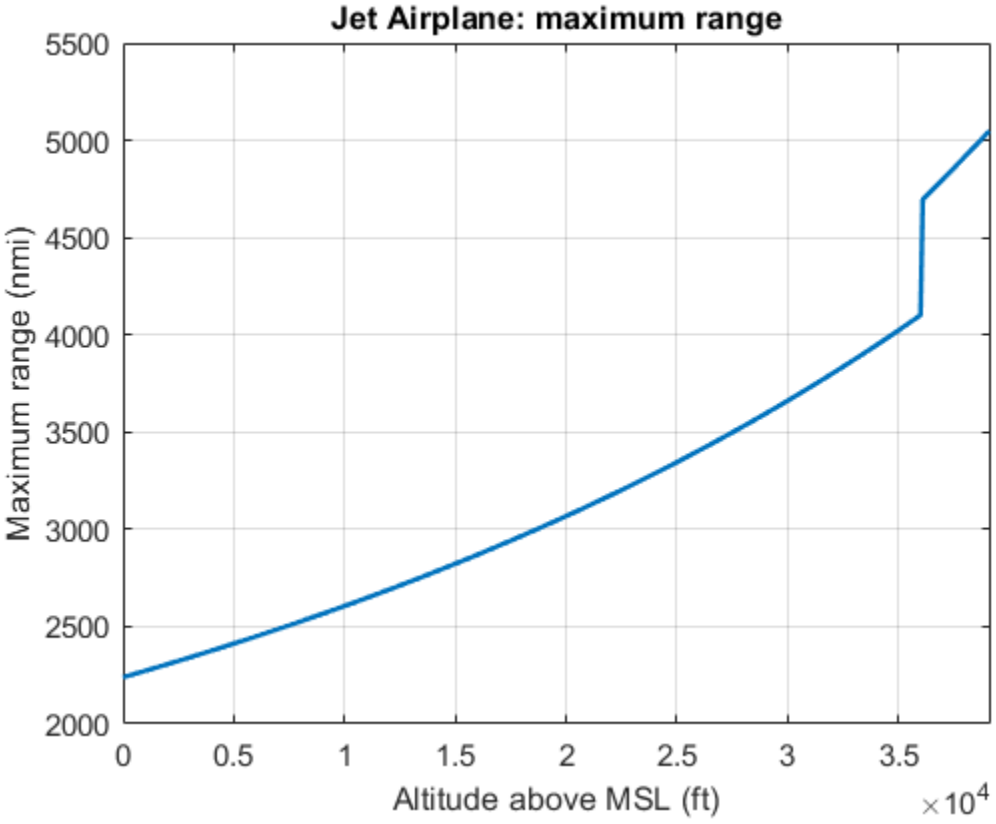

Jet Airplane: maximum rate of climb
Maximum rate of climb (ft/min)
9000
8000
7000
6000
5000
4000
3000
2000
1000
0
0
0.5
1
1.5
2
2.5
3
3.5
4
Altitude above MSL (ft)
×10^4

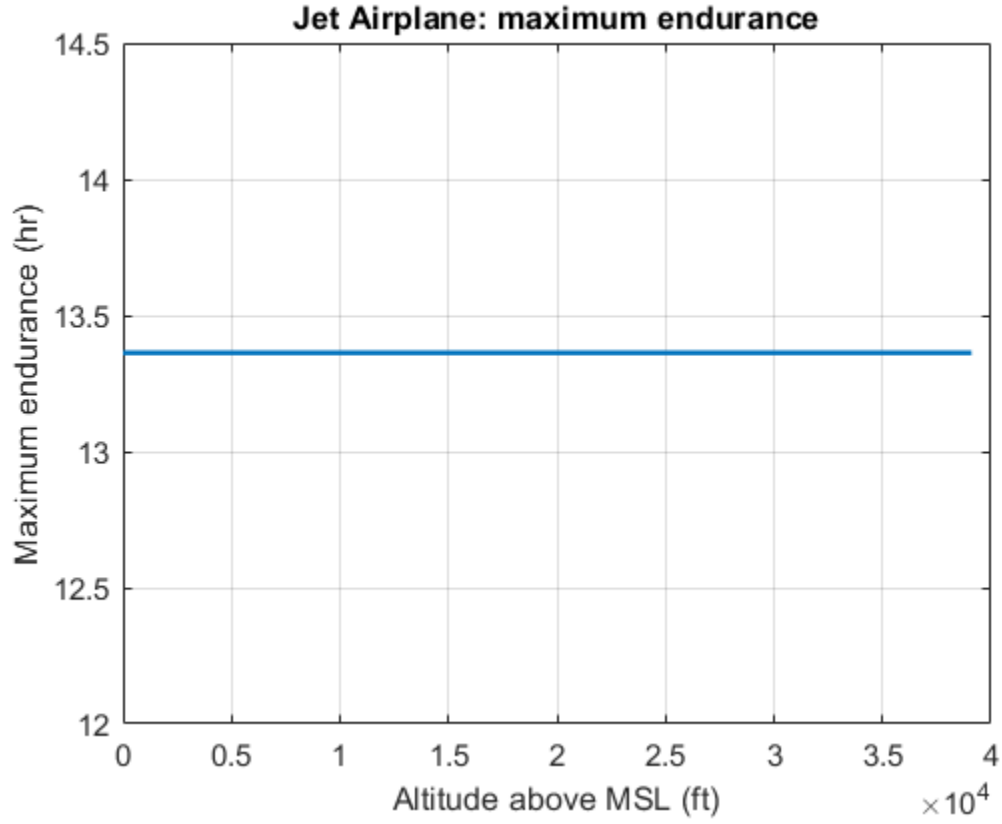

Jet Airplane: maximum turn rate
Maximum turn rate (deg/s)
10
9
8
7
6
5
4
3
2
1
0
0.5
1
1.5
2
2.5
3
3.5
4
Altitude above MSL (ft)
×10^4

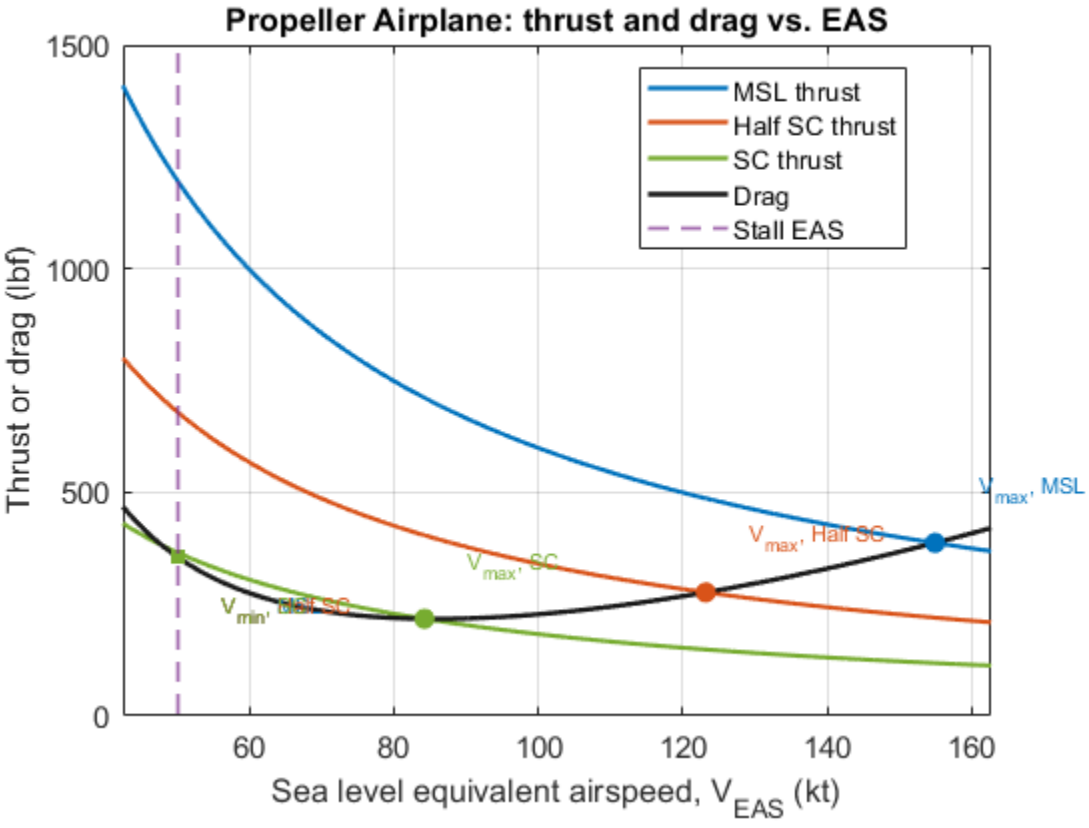

Jet Airplane: V-n diagram at MSL
Load factor, n
True airspeed, V_{TAS} (kt)
Positive envelope
Negative envelope
Maximum airspeed
Positive n_{max}
Negative n_{max}
1.5
1
0.5
0
-0.5
-1
-1.5
0
100
200
300
400
500

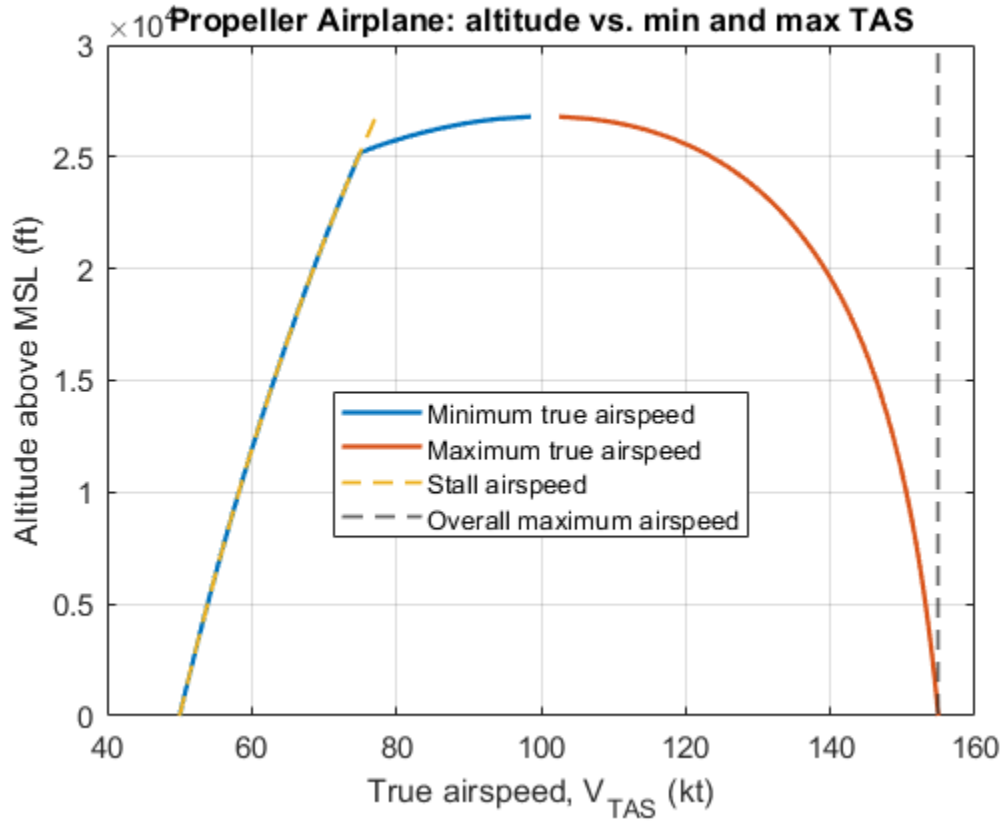

Jet Airplane: V-n diagram at Half SC
Load factor, n
True airspeed, V_{TAS} (kt)
Positive envelope
Negative envelope
Maximum airspeed
Positive n_{max}
Negative n_{max}
1.5
1
0.5
0
-0.5
-1
-1.5
0
100
200
300
400
500

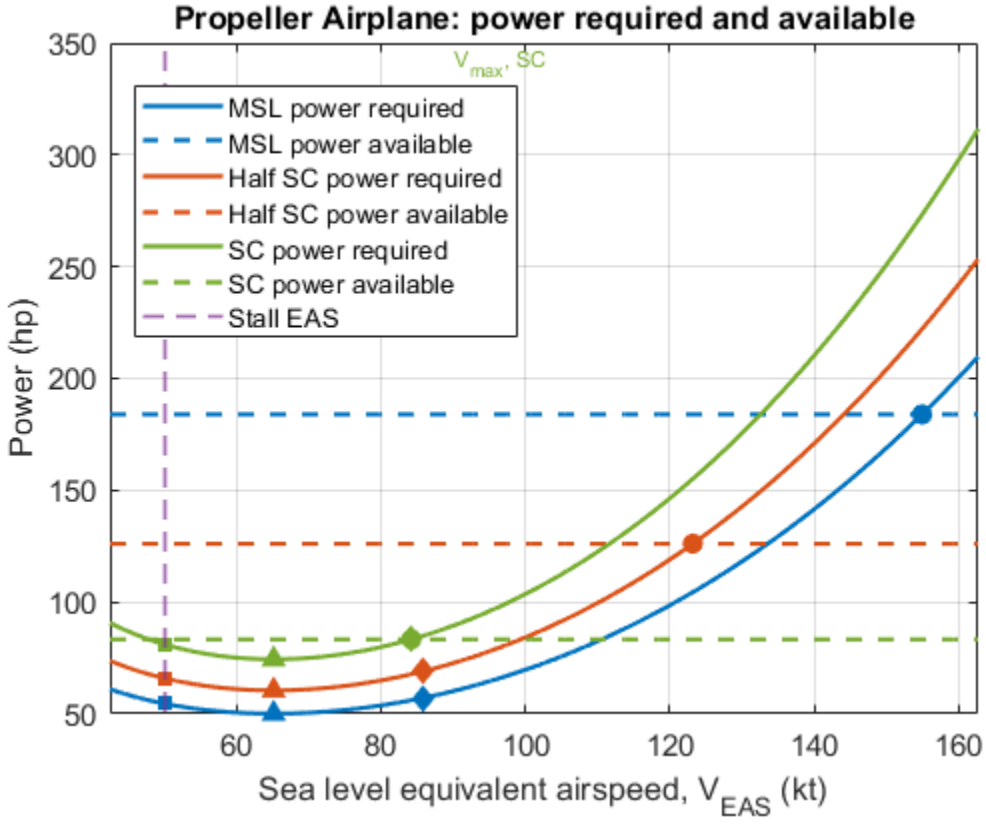

Jet Airplane: V-n diagram at SC
Load factor, n
True airspeed, V_{TAS} (kt)
Positive envelope
Negative envelope
Maximum airspeed
Positive n_{max}
Negative n_{max}
1.5
1
0.5
0
-0.5
-1
-1.5
0
100
200
300
400
500

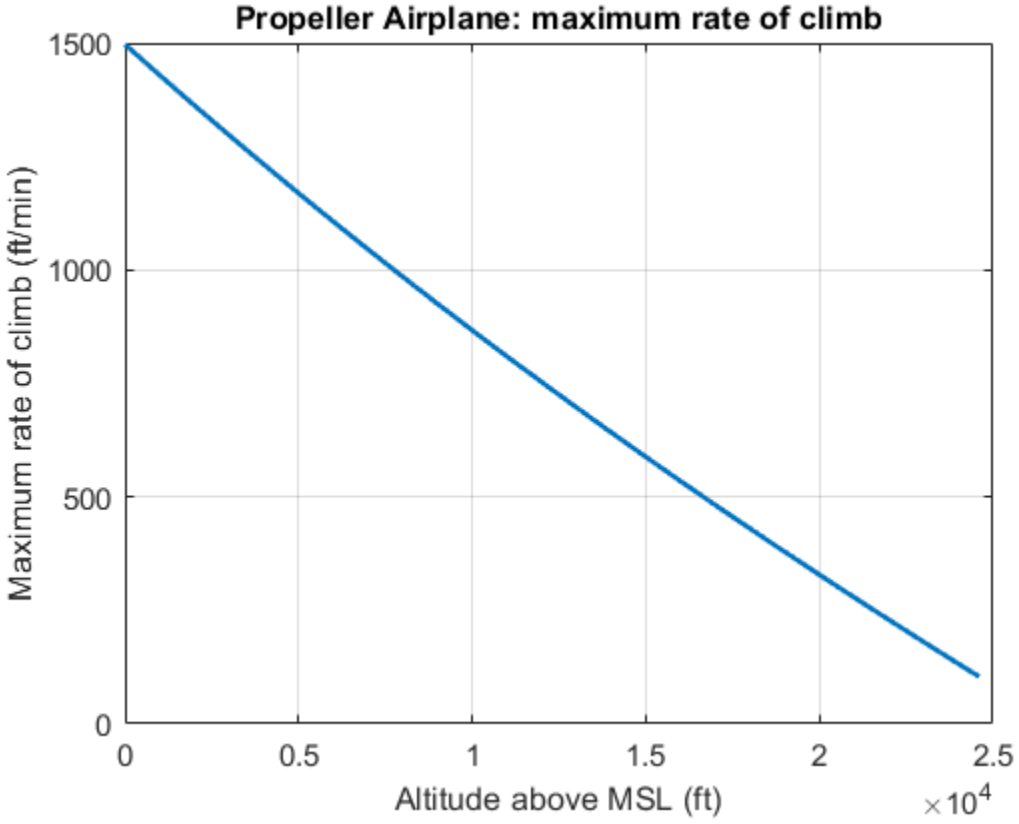

Jet Airplane: maximum range
Maximum range (nmi)
Altitude above MSL (ft)
$\times 10^4$
5500
5000
4500
4000
3500
3000
2500
2000
0
0.5
1
1.5
2
2.5
3
3.5

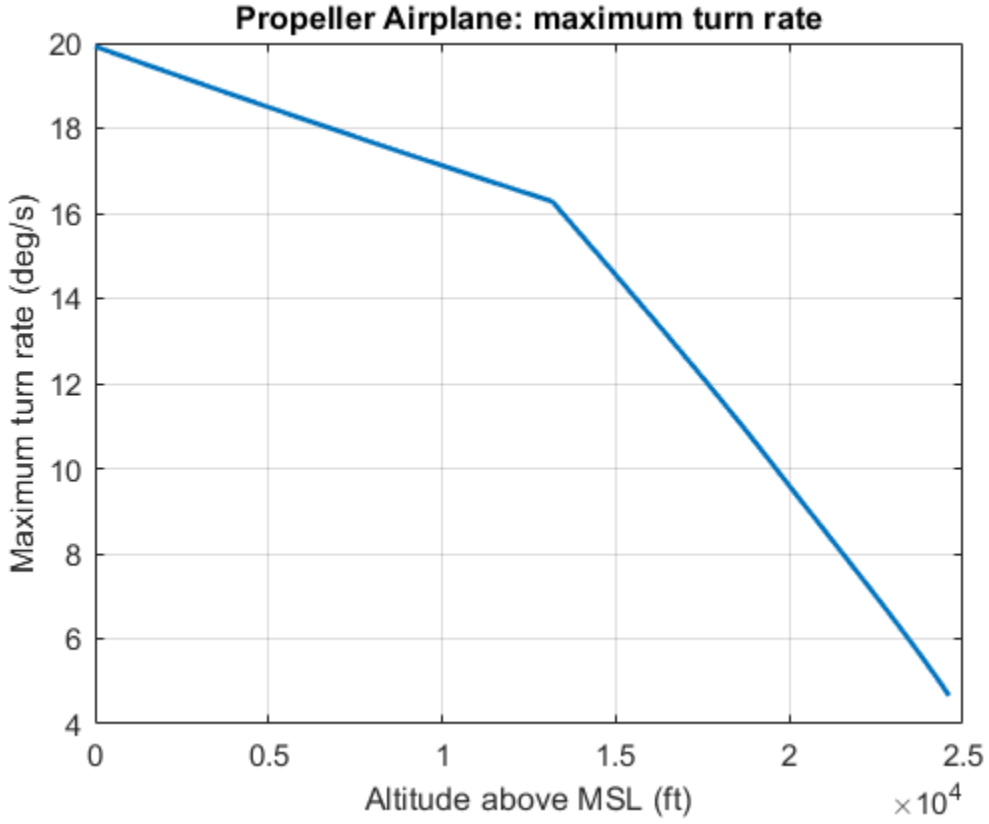

Jet Airplane: maximum endurance
Maximum endurance (hr)
14.5
14
13.5
13
12.5
12
0
0.5
1
1.5
2
2.5
3
3.5
4
Altitude above MSL (ft)
×10^4

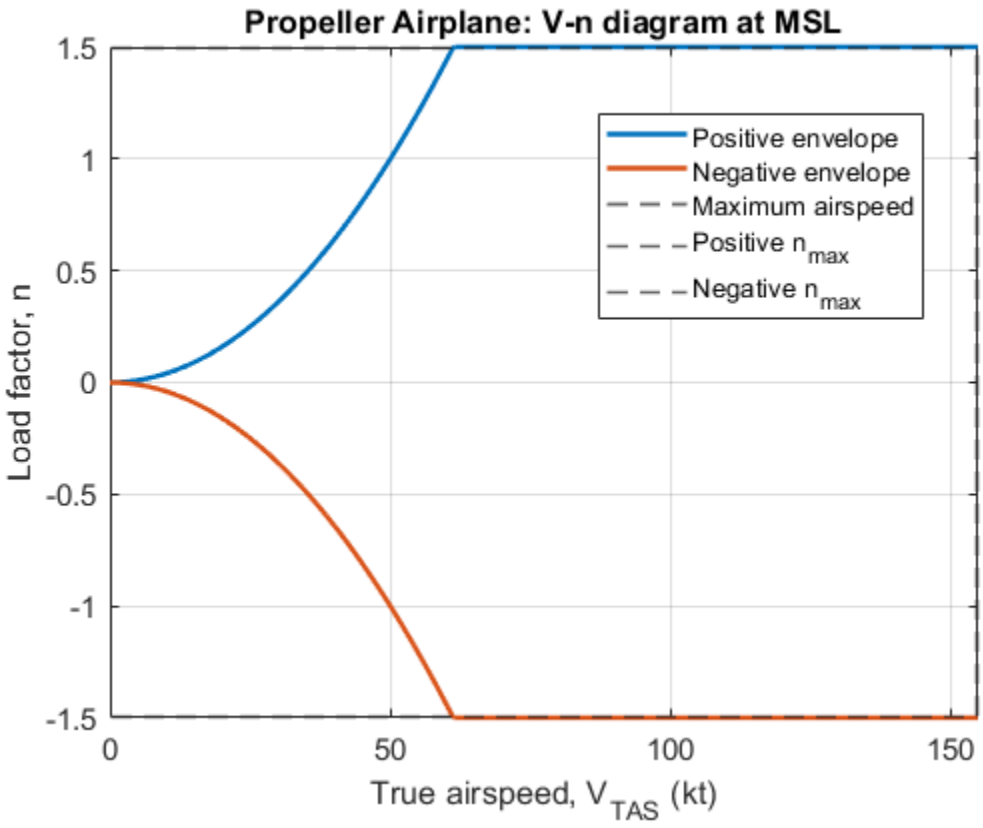

Propeller Airplane: thrust and drag vs. EAS
MSL thrust
Half SC thrust
SC thrust
Drag
Stall EAS
Thrust or drag (lbf)
1500
1000
500
0
60
80
100
120
140
160
Sea level equivalent airspeed, V_EAS (kt)
V_max, MSL
V_max, Half SC
V_max, SC
V_min, MSL SC

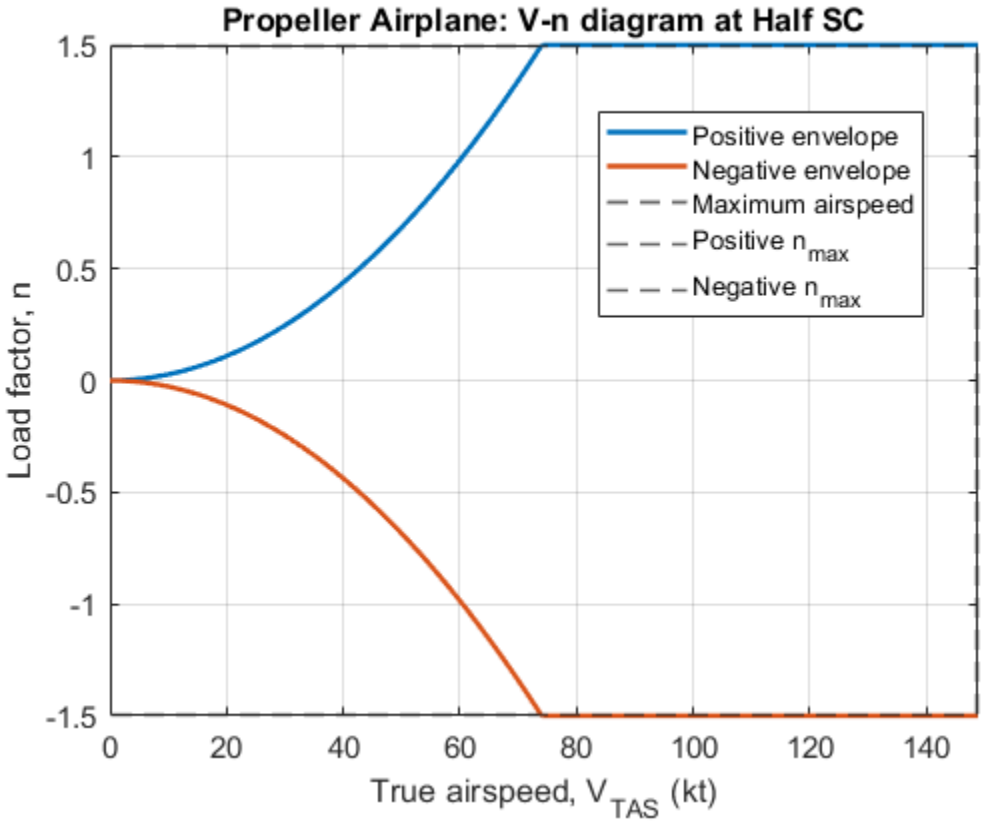

Propeller Airplane: altitude vs. min and max TAS
×10^4
3
2.5
2
1.5
1
0.5
0
Altitude above MSL (ft)
40
60
80
100
120
140
160
True airspeed, V_TAS (kt)
Minimum true airspeed
Maximum true airspeed
Stall airspeed
Overall maximum airspeed

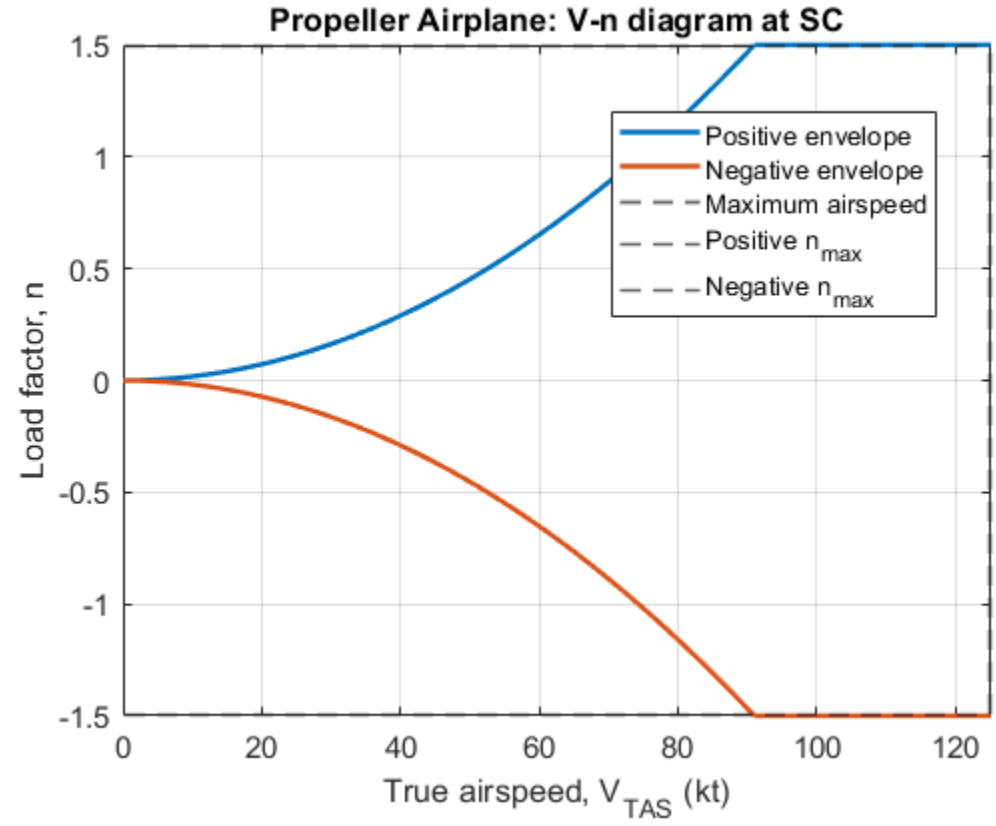

Propeller Airplane: power required and available
V_max, SC
MSL power required
MSL power available
Half SC power required
Half SC power available
SC power required
SC power available
Stall EAS
350
300
250
200
150
100
50
Power (hp)
60
80
100
120
140
160
Sea level equivalent airspeed, V_EAS (kt)

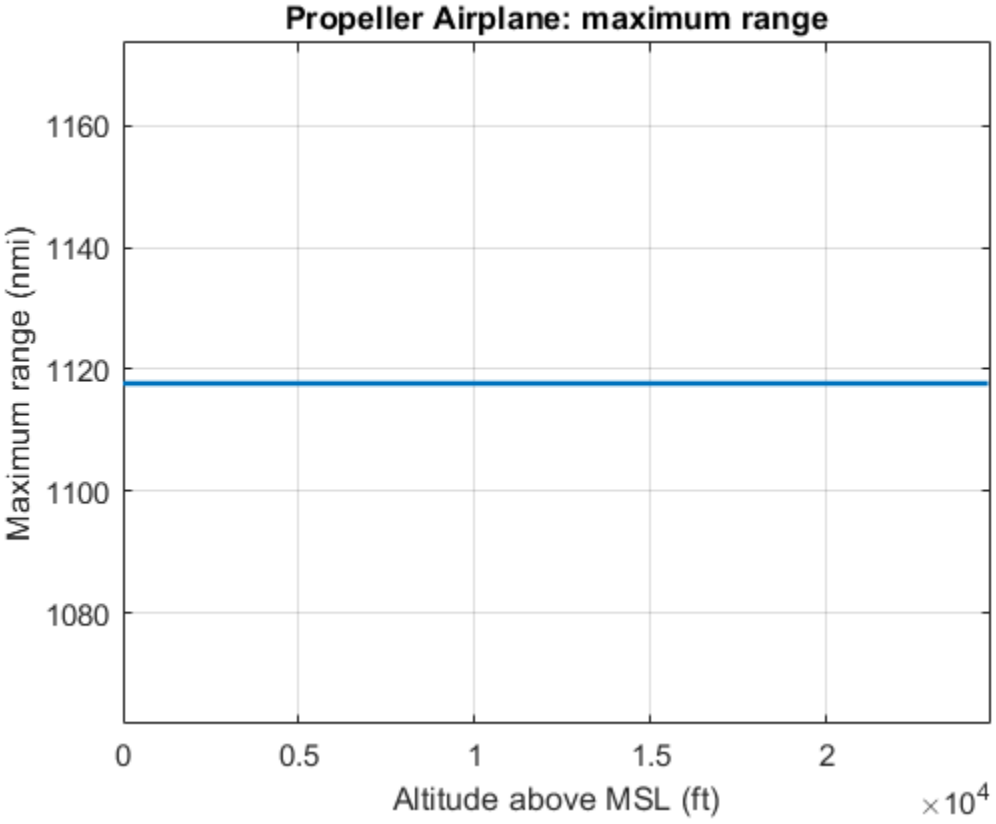

Propeller Airplane: maximum rate of climb
Maximum rate of climb (ft/min)
1500
1000
500
0
0
0.5
1
1.5
2
2.5
Altitude above MSL (ft)
×10^4

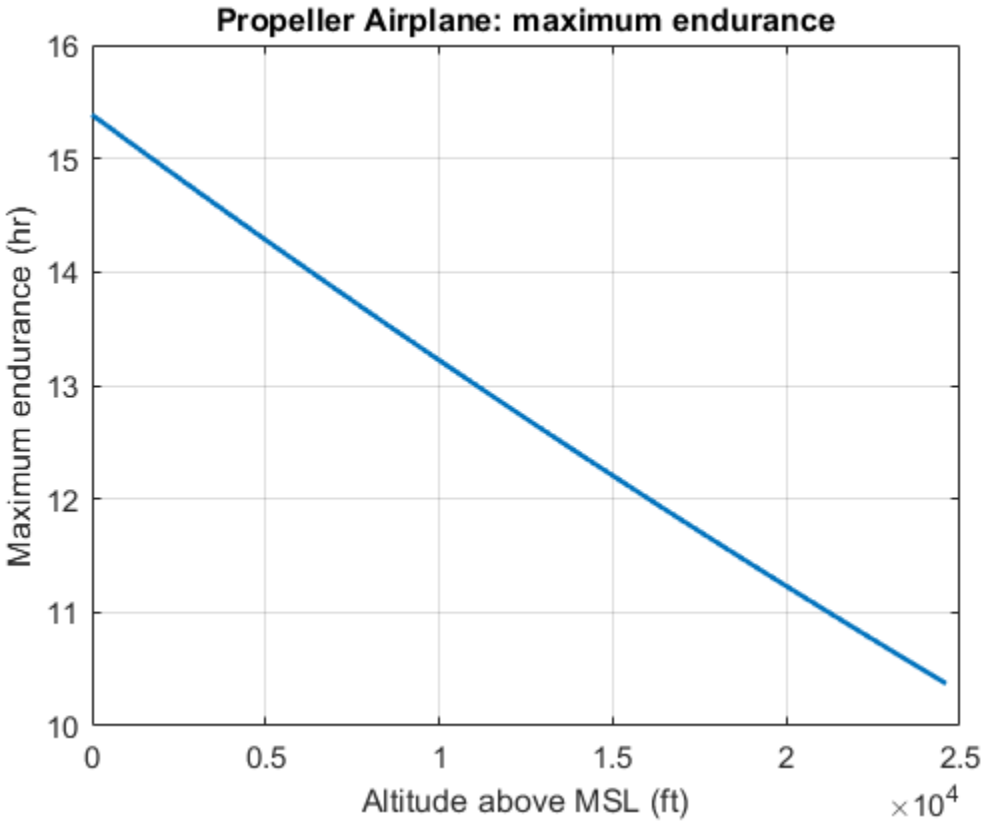

Propeller Airplane: maximum turn rate
Maximum turn rate (deg/s)
20
18
16
14
12
10
8
6
4
0
0.5
1
1.5
2
2.5
Altitude above MSL (ft)
×10^4


Propeller Airplane: V-n diagram at MSL
Load factor, n
True airspeed, V TAS (kt)
Positive envelope
Negative envelope
Maximum airspeed
Positive n max
Negative n max
1.5
1
0.5
0
-0.5
-1
-1.5
0
50
100
150


Propeller Airplane: V-n diagram at Half SC
Load factor, n
True airspeed, V TAS (kt)
Positive envelope
Negative envelope
Maximum airspeed
Positive n max
Negative n max
1.5
1
0.5
0
-0.5
-1
-1.5
0
20
40
60
80
100
120
140


Propeller Airplane: V-n diagram at SC
Positive envelope
Negative envelope
Maximum airspeed
Positive n max
Negative n max
Load factor, n
True airspeed, V TAS (kt)
1.5
1
0.5
0
-0.5
-1
-1.5
0
20
40
60
80
100
120


Propeller Airplane: maximum range
Maximum range (nmi)
Altitude above MSL (ft)
×10^4
1160
1140
1120
1100
1080
0
0.5
1
1.5
2



AI Use Statement

For this assignment's preparation, the author utilized ChatGPT, a language model created by OpenAI. Within this assignment, ChatGPT was used for brainstorming, checking MATLAB code organization, identifying possible errors, polishing code, and suggesting debugging fixes.

Published with MATLAB® R2023b